

Relatório 5 - Prática de React e Next.js

Kaíque Medeiros Lima

1. Descrição da atividade

a. Componentes e organização do gerenciador de tarefas

A primeira parte da atividade é um gerenciador de tarefas em React. A aplicação foi dividida em componentes como AddTask, Tasks e Title, além de elementos reutilizáveis para os campos e botões. O App reúne o estado principal e passa as informações e funções para os componentes pelas props. Assim, cada parte da interface fica com uma responsabilidade mais definida.

O AddTask controla os campos de título e descrição com useState. Antes de adicionar uma tarefa, o código verifica se os dois campos foram preenchidos, usando trim para não aceitar só espaços. Depois da inclusão, os campos são limpos. A tarefa recebe um identificador com crypto.randomUUID e começa com isCompleted igual a false.

b. Estado, eventos e armazenamento local

As ações da lista mostram como o estado acompanha a interação do usuário. Para marcar uma tarefa como concluída, o código usa map e troca o valor de isCompleted apenas no item escolhido. Para excluir, usa filter e mantém os itens com identificadores diferentes. Na inclusão, cria uma nova lista com as tarefas anteriores e o novo objeto.

O useEffect salva as tarefas no localStorage sempre que a lista muda, e o estado inicial recupera os dados usando JSON.parse. Com isso, as tarefas podem continuar disponíveis ao recarregar a página no mesmo navegador. Também existe um exemplo de busca de tarefas com fetch, mas a chamada dessa função está comentada. Então, a versão registrada trabalha com a lista local, sem carregar automaticamente as tarefas da API.

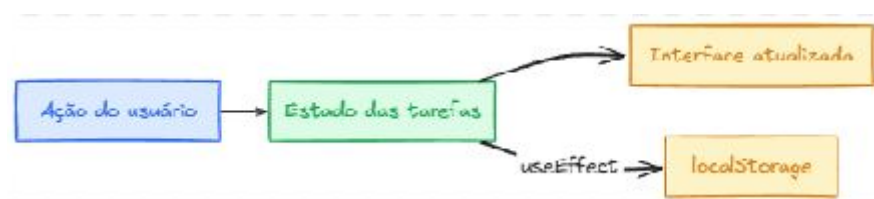


Figura 1 - Estado, interface e persistência das tarefas. Autoria própria, Excalidraw.

c. Navegação e apresentação dos detalhes

O projeto utiliza o React Router para separar a página inicial da rota /task. Quando o usuário abre os detalhes, o título e a descrição são colocados na URL por meio de URLSearchParams. Na outra página, useSearchParams recupera esses valores e apresenta o conteúdo da tarefa. O botão de voltar usa a navegação do histórico.

A interface usa classes do Tailwind CSS e ícones do lucide-react. A tarefa concluída recebe um ícone de confirmação e o texto riscado, enquanto os botões de detalhes e

exclusão ficam ao lado de cada item. Esses elementos deixam as ações mais fáceis de identificar e conectam a mudança do estado com uma resposta visual.



Figura 2 - Lista de tarefas, exemplo da aula. Curso de React para Completos Iniciantes [2026].

d. Miniblog com Next.js e organização por módulos

A segunda parte é um miniblog sobre a temporada de 2021 da Fórmula 1, desenvolvido com Next.js e TypeScript. A pasta `app` organiza as rotas, enquanto os módulos `blog`, `auth` e `profile` reúnem as telas e os recursos de cada funcionalidade. Componentes como `PageShell`, `SiteHeader`, `SiteFooter`, `TextField` e `PrimaryButton` ficam em `shared` para serem reaproveitados.

A página inicial apresenta dois artigos definidos em um arquivo local. A interface `Article` descreve campos como `slug`, `título`, `descrição`, `categoria`, `data` e `conteúdo`. Na rota dinâmica `/artigos/[slug]`, o código procura o artigo correspondente e chama `notFound` quando ele não existe. O `generateStaticParams` fornece os `slugs` cadastrados para a geração das páginas.

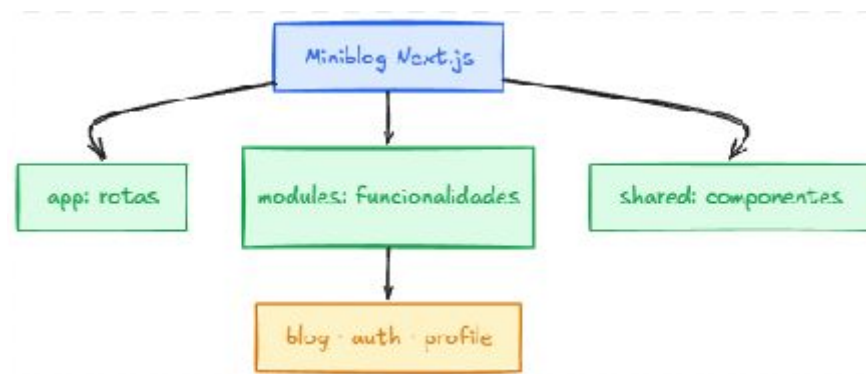


Figura 3 - Organização do miniblog por responsabilidade. Autoria própria, Excalidraw.

e. Hooks e interação no miniblog

A lista de artigos usa `useState` para guardar o texto da busca e filtra os títulos sem diferenciar letras maiúsculas e minúsculas. Quando não encontra nenhum resultado, apresenta uma mensagem no lugar da lista. O `useRef` guarda a referência do campo de busca, permitindo que o botão Focar direcione o cursor para ele.

O `useEffect` altera o título da página enquanto a lista está montada e define uma função de limpeza para restaurá-lo. As partes que usam estado e eventos estão marcadas com `use client`, como a lista, os formulários e o perfil. Já os arquivos das páginas delegam essa interação às telas, mantendo a organização entre a estrutura das rotas e o comportamento dos componentes.

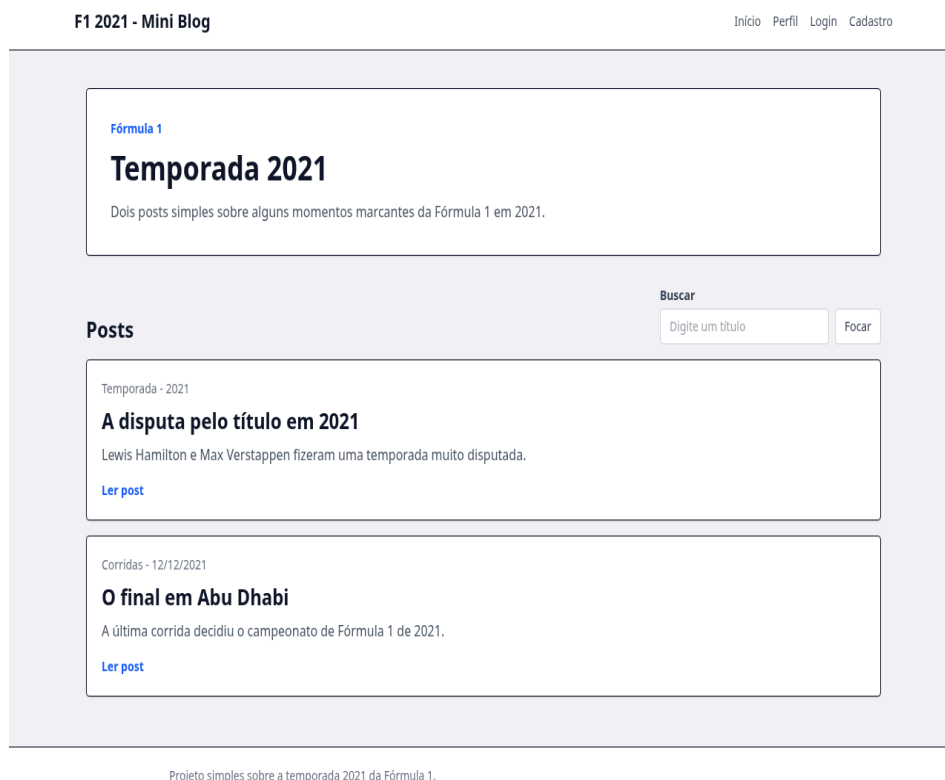


Figura 4 - Miniblog, tema: F1. Autoria própria.

f. Formulários, perfil e limites da prática

As telas de login e cadastro reutilizam os mesmos campos e botões. Os formulários usam campos obrigatórios e tipos como email e password, no envio, impedem o recarregamento e exibem uma mensagem. Essa etapa é uma demonstração de interface, sem autenticação real nem gravação de usuários em um banco.

O perfil permite editar um nome e atualizar o texto de nome salvo com `useState`. Esse valor fica no estado do componente, sem persistência após o recarregamento. Da mesma forma, o grupo de rotas chamado (secure) organiza os arquivos, mas seu layout apenas devolve o conteúdo recebido e não faz controle de acesso. Os exemplos ajudam a separar a construção da tela das regras de segurança e armazenamento que uma aplicação completa precisaria ter.

f. O que é Next.js? Uma Análise do Popular Framework JavaScript

O artigo apresenta o Next.js como um framework baseado em React voltado para a construção de aplicações web prontas para produção. Além dos componentes do React, o framework acrescenta recursos próprios para organização das páginas, roteamento, renderização e otimização da aplicação. O texto também destaca o suporte a TypeScript e diferentes formas de renderização, como SSR, geração estática e renderização no cliente.

Um dos pontos principais apresentados é o sistema de rotas baseado na estrutura de arquivos. O artigo mostra como páginas e pastas podem representar diferentes URLs, incluindo rotas fixas, aninhadas e dinâmicas. Nas rotas dinâmicas, valores como um identificador ou slug passam a fazer parte do caminho da página.

O texto também aborda a pré-renderização e explica a diferença entre gerar uma página antecipadamente e renderizar a cada request. Na geração estática, os dados podem ser preparados durante a construção da aplicação e a página resultante pode ser reutilizada. Na prática realizada, o *generateStaticParams* fornece previamente os slugs dos artigos disponíveis, relacionando a estrutura do projeto com essa possibilidade de geração das páginas.

2. Conclusões

As duas práticas mostram como os componentes e os hooks trabalham juntos para montar uma interface. No gerenciador, as ações atualizam a lista e o armazenamento local. O Next.js acrescenta a organização das páginas e das rotas, enquanto o TypeScript deixa o formato dos dados e das props mais explícito. A separação entre módulos e componentes compartilhados também aproxima a prática das boas práticas de arquitetura, porque facilita entender cada parte e reaproveitar o que aparece em várias telas.

3. Referências

<https://www.youtube.com/watch?v=e6FigV2fLC8&t=77s>

<https://www.youtube.com/watch?v=2RWsLmu8yVc&t=237s>

<https://kinsta.com/pt/blog/next-js/>